

SPOT Quick Start Guide

This quick start guide is meant to provide help to speed up the learning process required for starting to use the SPOT tool. Because of this it will not cover all functionalities but only the very basic ones. The details and tricks of the more complex features can be discovered and learned over time, only if necessary. After all, this tool should be simple to learn and to use, as the name suggests.

1. Define a pseudo orchestration scenario

The best learning experience is by doing it, which is why this guide will follow an example from scratch. Normally this part, the development of the step scripts/functions, is done by the user and the SPOT tool begins to be used afterwards. Let us consider the following example:

- Simulate data processing on both local and remote systems by using a function to generate custom objects;
- Inside the above function, get a random number and check if it is odd or not – this will represent the detection of a condition;
- In parallel, locally, generate some other custom object to represent processing some data
- Simulate processing the results by merging the generated objects from all steps and exporting them to a local file;
- Simulate the use of conditions by using the above condition detection on a step;
- Include OrchVar dependent code inside the step scripts that will cause errors, to simulate also the use of OrchVars inside scripts/functions and also the error handling;

Requirements:

- One remote computer available besides the local one from where SPOT is executed;
- Credentials available for the remote computer;
- Administrative rights on the local and remote computer;

2. Install SPOT

There are two options to install SPOT on a Windows Computer: using the PowerShell default repository (which allows only online installations) or using the installation script “SPOT-Installer.ps1” (which allows both online and offline installations).

Repository method

For the method using the PowerShell Gallery repository, the online Microsoft documentation is available here: <https://learn.microsoft.com/en-us/powershell/gallery/getting-started?view=powershellget-3.x>

As mentioned in the documentation above, for installing SPOT, the command would be:

```
Install-Module -Name SPOT
```

The SPOT module dependencies would be installed automatically and the remaining SPOT tool dependencies (SshNet and PsExec) would have to be installed by using the following SPOT command:

```
Expand-SPOTCapability -Online
```

At the end, check that everything is installed correctly with the following command (the “SPOT Status” and the “SPOT Secret Store” may not be already initialized and at this stage, this is normal):

```
Show-SPOTDetails
```

```
2025-12-26 04:45:56 :: ===== Starting function Show-SPOTDetails =====
```

```
2025-12-26 04:45:56 :: SPOT version      : 1.0.0
```

```
2025-12-26 04:45:56 :: SPOT Status      : Initialized
```

```
2025-12-26 04:45:56 :: SPOT Capability  : Extended
```

```
2025-12-26 04:45:57 :: SPOT Secret Store : Initialized
```

```
2025-12-26 04:45:57 :: PsExec version   : 2.43
```

```
2025-12-26 04:45:57 :: SshNet version   : 2024.0.0.0
```

```
2025-12-26 04:45:57 :: ===== Finished function Show-SPOTDetails =====
```

Script Installer method

For the installer method, the installer script “SPOT-Installer.ps1” provided with SPOT may be used.

There are 2 modes of installation with this method, depending on the conditions:

- **Install From Internet**, which is also an online installation method, but with the difference that all SPOT dependencies (modules and tools) are installed. The command to use would be:
 - `.\SPOT-Installer.ps1 -InstallFromInternet`
- **Install Offline**, which is done in 2 steps as follows:
 - Download all SPOT related packages to create a big, offline, SPOT thick package.
 - `.\SPOT-Installer.ps1 -CreateSPOTPackage -SPOTPackagePath C:\temp\SPOT.zip`
 - Install offline from the thick package provisioned in the first step.
 - `.\SPOT-Installer.ps1 -InstallFromSPOTPackage -SPOTPackagePath C:\temp\SPOT.zip`

After the SPOT installation, in order to use the tool properly and create projects, SPOT must also be initialized. This initialization is performed on two aspects:

- SPOT status initialization, which is done at the OS level and makes sure that the main OS requirements are configured to support SPOT. The command to use for this initialization is:
 - o `Initialize-SPOT -MasterPassword MyPassword`
- SPOT Secret Store initialization, which is done at the Secret Store PowerShell module level, to have it configured to support SPOT. The command to use for this initialization is:
 - o `Initialize-SPOTSecretStore`

Of course, the password used above is just an example, but what is important is that it must be typed in in clear text. This master password supplied now is placed in encrypted form inside an environment variable of the current user. There is also an option to never prestage this master password locally while instead ask it each time it is needed, but for the moment, we use the default prestage password option.

3. Create a SPOT project

After the SPOT initialization is done, projects can be safely created and worked upon. The command to use to create a project from scratch is:

```
Initialize-SPOTProject -ProjectPath C:\QuickStartProject
```

This command created the project folder structure required for a minimal SPOT project.

The details regarding specific subfolders or files from a project folder will be revealed soon, in the next sections of this quick start guide and will be easier to understand this way.

3.1 Develop the step scripts/functions

This is the part that is already mostly done or will be done by the user. That is because, as said, SPOT does not provide a collection of orchestration functions to accomplish various orchestration tasks, but rather just orchestrates such user provided scripts and functions. In this chapter, we will create some orchestration scripts and functions while explaining some key SPOT related elements inside them.

The scripts

The scripts are placed in the “_Scripts” folder. They can have parameters or not. For this exercise we create two scripts.

Generate-TestData.ps1

```
#####  
Param (  
    [Parameter(Mandatory=$false)]  
    [object[]]
```

```

ones      $InputObjects, # the objects from outside, to be merged with the generated
[Parameter(Mandatory=$true)]
[int]
$ObjectCount, # the number of test objects generated
[Parameter(Mandatory=$true)]
[String]
$ObjectClass # The value of the Class attribute from each object generated
)

#####
# log the script start
Write-ProjectLog "Starting the script Generate-TestData."

#####
# generate some objects
$returnObjects = Generate-Objects -ObjectCount $ObjectCount -ObjectClass
$ObjectClass

#####
# insert the input objects, if any
if ($null -ne $InputObjects) {
    $returnObjects += $InputObjects
}

#####
# generate some random condition variable inside the current script, to be checked
later by other orchestration steps, potentially on other computers
if ((Get-Random)%2) {
    $TestCondition = $true
    Write-ProjectLog "Odd number generated. Random TestCondition is
$TestCondition."
}
else {
    $TestCondition = $false
    Write-ProjectLog "Even number generated. Random TestCondition is
$TestCondition."
}

#####
# perform test error handling depending on some OrchVar
if ($OrchVars.TestVariable1 -eq "NTE") {
    Write-ProjectLog "ERROR: OrchVar TestVariable1 detected with
NonTerminatingError flag."
    Write-Error "Simulated nont-terminating error triggered by TestVariable."
}
elseif ($OrchVars.TestVariable1 -eq "TE") {
    Write-ProjectLog "ERROR: OrchVar TestVariable1 detected with TerminatingError
flag."
    throw "Simulated terminating error triggered by TestVariable."
}

#####
# perform controlled stop of the script with signaling that something went wrong
if ($OrchVars.TestVariable2 -eq "BreakControl") {
    Write-ProjectLog "OrchVar TestVariable2 detected with BreakControl flag.
Returning false."
    # stop the current script and the runbook in a controlled manner, if the
situation is not as expected
    throw "Generate-TestData: BreakControl flag detected!"
}

#####
# generate some other objects
$returnObjects += Generate-Objects -ObjectCount $ObjectCount -ObjectClass
"SecondSet-$ObjectClass"

#####
# log the script end
Write-ProjectLog "Finished the script Generate-TestData."
#####

```

Export-GeneratedData.ps1

```
#####
Param (
    [Parameter(Mandatory=$true)]
    [object[]]
    $InputObjects, # the objects from outside, to be included in the exported
    [Parameter(Mandatory=$true)]
    [int]
    $ObjectCount, # the number of test objects generated by this script
    [Parameter(Mandatory=$true)]
    [String]
    $ObjectClass, # The value of the Class attribute from each object generated by
this script
    [Parameter(Mandatory=$true)]
    [String]
    $ExportPath # The path of the export file
)

#####
# log the script start
Write-ProjectLog "Starting the script Export-GeneratedData."

#####
# generate some objects
Write-ProjectLog "Generating a set of objects locally."
$ExportObjects = Generate-Objects -ObjectCount $ObjectCount -ObjectClass
$ObjectClass

#####
# add the objects from outside
Write-ProjectLog "Add to the local objects the object array from outside."
$ExportObjects += $InputObjects

#####
# do the actual export
Write-ProjectLog "Perform the export."
$ExportObjects | Export-Csv -Path $ExportPath -Delimiter ";" -NoTypeInfoation

#####
# log the script end
Write-ProjectLog "Finished the script Export-GeneratedData."
#####
```

Some observations about these scripts:

- They contain some custom called functions, "Generate-Objects" and "Write-ProjectLog". These functions are not defined inside the script because they will be define inside a separate helper functions file. This is for the purpose of reusing, without defining them in each script.
- To signal that a script has reached a situation where the entire orchestration should stop, a terminating error needs to be generated (e.g. with the "throw" command). This is normal PowerShell error handling. When a script does not end with a terminating error, it is considered completed successfully.
- Some scripts contain references to undeclared variables, like this one "\$OrchVars.TestVariable1". The \$OrchVars variable will be prepopulated at execution time with values from a central SPOT configuration repository, which are basically values loaded from a yaml file, OrchVars.yaml.

The helper functions file

The helper functions files are placed in the “_HelperFunctions” folder. They should only contain function definitions, without any execution statements, because they will be loaded with dot sourcing and this is equivalent with an execution of the code. The names of these files are not important as they will not be referenced by name anywhere. SPOT will just load all files found in this folder, to access the functions defined inside them.

hFunctions.ps1

```
#####

function Write-ProjectLog {
    Param (
        [Parameter(Mandatory=$true)]
        [ValidateNotNullOrEmpty()]
        [String]
        $Message, # The log entry to be written
        [Parameter(Mandatory=$false)]
        [String]
        $LogPath = "C:\temp\Project.log" # The path of the log file generated by the
current script
    )

    # Check existence of the temp folder and create it if it is not there
    if (!(Test-Path -Path (Split-Path -Path $LogPath -Parent) -PathType Container)) {
        New-Item (Split-Path -Path $LogPath -Parent) -Force -ItemType Directory -
Confirm:$false | Out-Null
    }

    # Format Date for the log File
    $FormattedDate = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

    # write the log entry to the normal output
    Write-Output "$FormattedDate :: $Message"

    # Write log entry to $LogPath
    "$FormattedDate :: $Message" | Out-File -FilePath $LogPath -Append -Encoding ascii
} # end of Write-ProjectLog function

function Generate-Objects {
    Param (
        [Parameter(Mandatory=$true)]
        [int]
        $ObjectCount, # the number of test objects generated
        [Parameter(Mandatory=$true)]
        [String]
        $ObjectClass # The value of the Class attribute from each object generated
    )

    #####
    # generate the data
    $GeneratedObjects = @()
    $i = 1
    while ($i -le $ObjectCount) {
        # generate the object
        $GeneratedObjects += [PSCustomObject]@{
            Class = $ObjectClass;
            Iteration = $i;
            Computername = $env:COMPUTERNAME
            RandomNumber = Get-Random;
        }
        # wait to simulate load
        Start-Sleep -Seconds 1
        # increment
        $i++
    }
}
```

```

    }
    return $GeneratedObjects
} # end of Generate-Objects function

function Get-WinOSDescription {
    $Description = (Get-WmiObject -Class Win32_OperatingSystem).Description
    Write-ProjectLog "The local computer description is: $Description."
} # end of Get-WinOSDescription function

function Set-WinOSDescription {
    Param (
        [Parameter(Mandatory=$true)]
        [ValidateNotNullOrEmpty()]
        [string]
        $Description # decription to be added to the local computer OS
    )

    $OSInfo = Get-WmiObject -Class Win32_OperatingSystem
    $OSInfo.Description = $Description
    $OSInfo.Put()
} # end of Set-WinOSDescription function

#####

```

Some observations about these functions:

- Some of them can be used also as step functions, especially if they do not perform a task and output special objects (even in that case, they would execute fine, but the returned objects are only saved as log file content). In this case, they will be referenced by their function name.
- Some of them contain called functions, which is normal. These called function can be located in other helper function files, as long as they are together and will be loaded together.

The configuration files

There are two SPOT configuration files, one for configuration values to be used inside the step functions or as SPOT step parameters (which includes step function parameters) called OrchVars.yaml and one for the definition of the secrets to be used in SPOT called SInputs.yaml. These files are created by the project initialization script, but are currently empty.

OrchVars.yaml

```

#####

TestVariable1: TestValue

TestVariable2: TestValue2

_Debug: true

```

#####

SInputs.yaml

#####

Credentials:

SrvCred2:

Username: .\admin

Password: MyPassword

Secrets:

EnablePass:

Value: EnablePassword

#####

Some observations about these files:

- Inside OrchVars.yaml there are entries starting with “_” and entries that do not. The entries starting with “_” are reserved for SPOT settings. In our example, we observe that only the “_Debug” SPOT setting is enabled. This allows for very detailed logging to be available with regards to SPOT internal mechanisms.
- Also inside OrchVars.yaml, the entries “TestVariable1” and “TestVariable2” are SPOT project configurations and they are available inside all step functions or scripts at execution time as elements from the \$OrchVars hashtable variable and as step function parameters or as step parameters. For example, “\$OrchVars.TestVariable1” will hold the value written in the OrchVars.yaml file.
- Inside SInputs.yaml there are defined two types of secrets: one Credential and one Secret. Unlike the OrchVars entries above, the SVars (Secret Variables) defined in this file are not available inside the step functions at execution time (for security reasons, to limit the exposure of the entire set of secrets on all orchestration targets), but rather only available inside SPOT as step function parameters or as step parameters. To reference an entry from this file, the following notations can be used, \$SV:SrvCred2.
- Of course, these are just sample secrets. In a real environment, they have to be replaced with actual values for that environment.
- Since the line indentations are very important inside yaml files, it is strongly recommended to use a capable text editor to handle them with ease (e.g. Notepad++ gets installed by default with the Script Installer method). This applies also to the runbook files below.
- The secrets file, SInputs.yaml, is sensitive with all the secrets inside written in clear text. Because of this, its presence after initial population of the Secrets store is not required. If the file remains in this location, it will be used to refresh the secrets inside the Secrets Store at each SPOT execution. If the file is removed, SPOT will simply not refresh the Secrets Store. To populate the

Secrets Store with the secrets defined in this project file on demand, the following command can be used (afterwards, the SInputs.yaml file can be removed or placed in a safer location):

- `Import-SPOTProjectSecrets -ProjectPath C:\QuickStartProject`

3.2 Define the real orchestration inside the SPOT project

This is the part where the step functions defined above are referenced inside sequence control files called runbook files. The most important fact about this sequence file is that it has defined a sequence order, execution methods and referenced parameters.

The runbook files

The runbook files are placed inside the folder “__SPOT_Runbooks”. For this exercise we create four runbook files.

QuickStart-Runbook1.yaml

#####

Description: QuickStart-Runbook1 for learning/testing

RunbookSteps:

Step1:

Type: PowershellCommandLocal

Description: Test script executed locally

Seq: 5

StepParameters:

CommandName: Generate-TestData

CommandParameters:

ObjectCount: 10

ObjectClass: LocalObject5

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep1

- TestCondition=TestConditionStep1

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step2:

Type: PowershellCommandRemote

Description: Test script executed remotely

Seq: 10

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep1

ObjectCount: 10

ObjectClass: RemoteObject

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep2

- TestCondition=TestConditionStep2

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step3:

Type: PowershellCommandRemoteSJ

Description: Test script executed remotely with Scheduled Jobs

Seq: 15

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep2

ObjectCount: 10

ObjectClass: RemoteObjectSJ

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep3

- TestCondition=TestConditionStep3

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step4:

Type: PowershellCommandRemoteWMI

Description: Test script executed remotely with WMI

Seq: 20

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep3

ObjectCount: 10

ObjectClass: RemoteObjectWMI

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep4

- TestCondition=TestConditionStep4

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step5:

Type: PowershellCommandRemotePsExec

Description: Test script executed remotely with PsExec

Seq: 25

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep4

ObjectCount: 10

ObjectClass: RemoteObjectPsExec

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep5

- TestCondition=TestConditionStep5

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step6:

Type: PowershellCommandLocal

Description: Test script locally

Seq: 30

StepParameters:

CommandName: Export-GeneratedData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep5

ObjectCount: 10

ObjectClass: LocalObject30

ExportPath: \$OV:_ProjectPath:\$OV\SPOTExportedObjects.csv

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step7:

Type: PowershellCommandLocal

Description: Test function from helper functions

Seq: 5

StepParameters:

CommandName: Get-WinOSDescription

VariablesToPublish: Description=OSDescription

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step8:

Type: PowershellCommandLocal

Description: Test function from helper functions

Seq: 10

StepParameters:

CommandName: Set-WinOSDescription

CommandParameters:

Description: SPOT

Conditions: \$PV:TestConditionStep1

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

#####

Some observations about this file:

- It is organized on steps, each representing a step function/script being executed on a remote target computer or locally.
- The order of execution is dictated by the "Seq" value from each step.
- Both scripts and functions are used as step functions. In the case of scripts, the script file name (without extension) becomes the function name. They are referenced in the "StepParameters > CommandName" elements.
- There are different "Types" of steps, each representing the method of local or remote execution (e.g. via PSRemoting, or Scheduled Jobs, or WMI, or PsExec). These values are the actual names of internal SPOT function used for these methods.
- Some variables from inside the step functions are published and used subsequently by other runbook steps. For example, inside the "Step1", under the "VariablesToPublish" element, there is this statement "TestCondition=TestConditionStep1". What this does is it takes the value of the "TestCondition" variable inside the step function from Step1 and it publishes it under a different name, "TestConditionStep1", inside a SPOT internal variable called "\$PublishedData". From here, this variable is used in a subsequent step, "Step8", under the "Conditions" element with the notation "\$PV:TestConditionStep1".
- There is an internal SPOT setting from the OrchVars used as a parameter value in "Step6" for the parameter ExportPath. The notation is this: "\$OV:_ProjectPath:\$OV\SPOTExportedObjects.csv". Unlike other situations, here we have a string that contains a portion referenced by an OrchVar value and which continues with other strings. Because of this, there is also an ending notation for delimiting the place where the Variable value should be inserted. Naturally, in these cases, only string variables are usable.

3.3 Execute the real orchestration in SPOT

Now that all the content required for the orchestration is in place inside the project we can start the actual runbook execution. This action will require access to the master password used when SPOT was initialized. This master password is already registered since the SPOT initialization done earlier, so it will not be required now. But it will be used to unlock the Secrets Store. There are two options to execute a SPOT runbook.

Command line execution

This method is best suited for CI/CD pipelines but it can be used on any task. Besides the fact that this is more versatile for unattended scenarios, it does have a small disadvantage – it does not provide any option to stop the execution on demand and to resume it.

To execute the SPOT runbook from our project, the following command can be used:

```
Start-SPOT -ProjectPath C:\QuickStartProject\ -MainRunbookName QuickStart-Runbook1
```

As it can be seen, the project is referenced by its folder path and the runbook is referenced by its file name (without the extension). During execution, the progress will be illustrated in the command line with periodic messages.

GUI execution

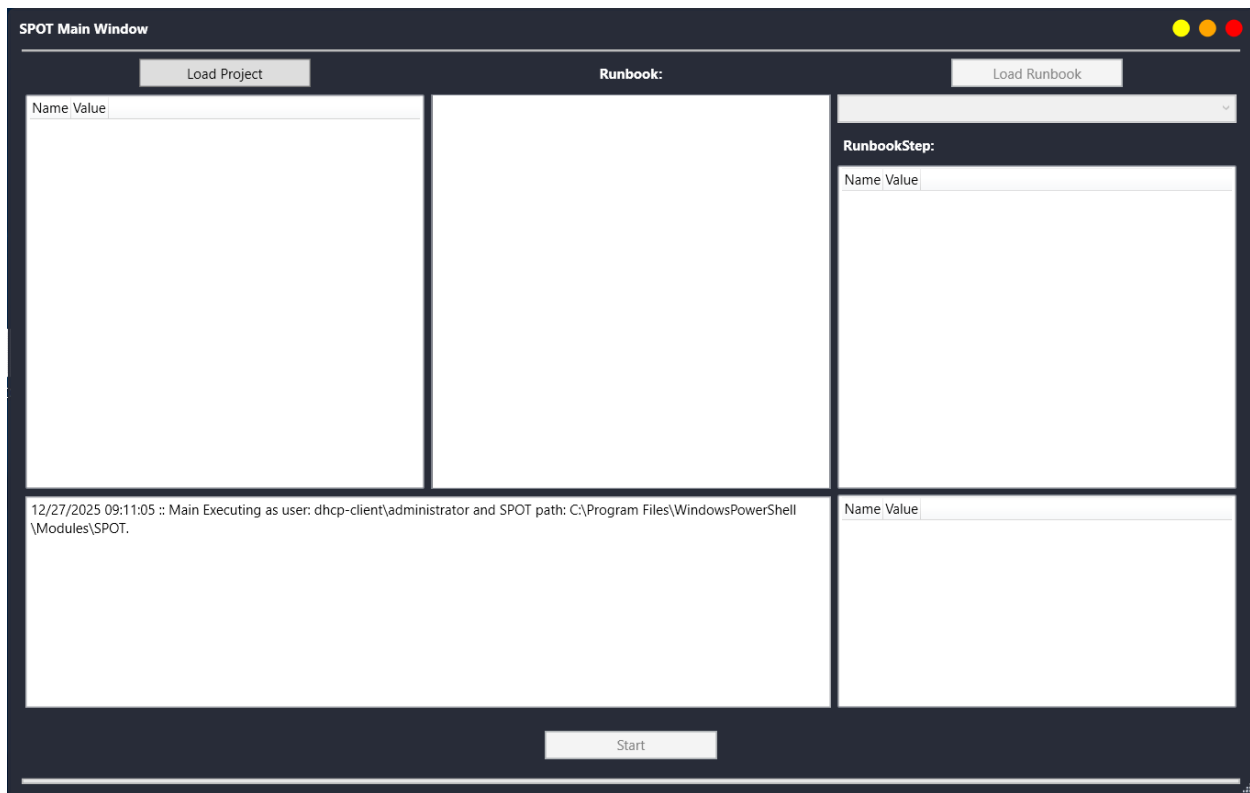
This method is best suited for assisted orchestrations, like deployments. Under the hood, the same functions are used to drive the orchestration. The difference is that they are wired into a GUI window. The main advantage for the GUI method has been already mentioned above, the possibility to stop and resume. In addition to this, the visualization of the runbook file structure and the progress report are much better.

There are no GUI methods to edit the runbook files. The GUI is made only for visualization of status and execution control.

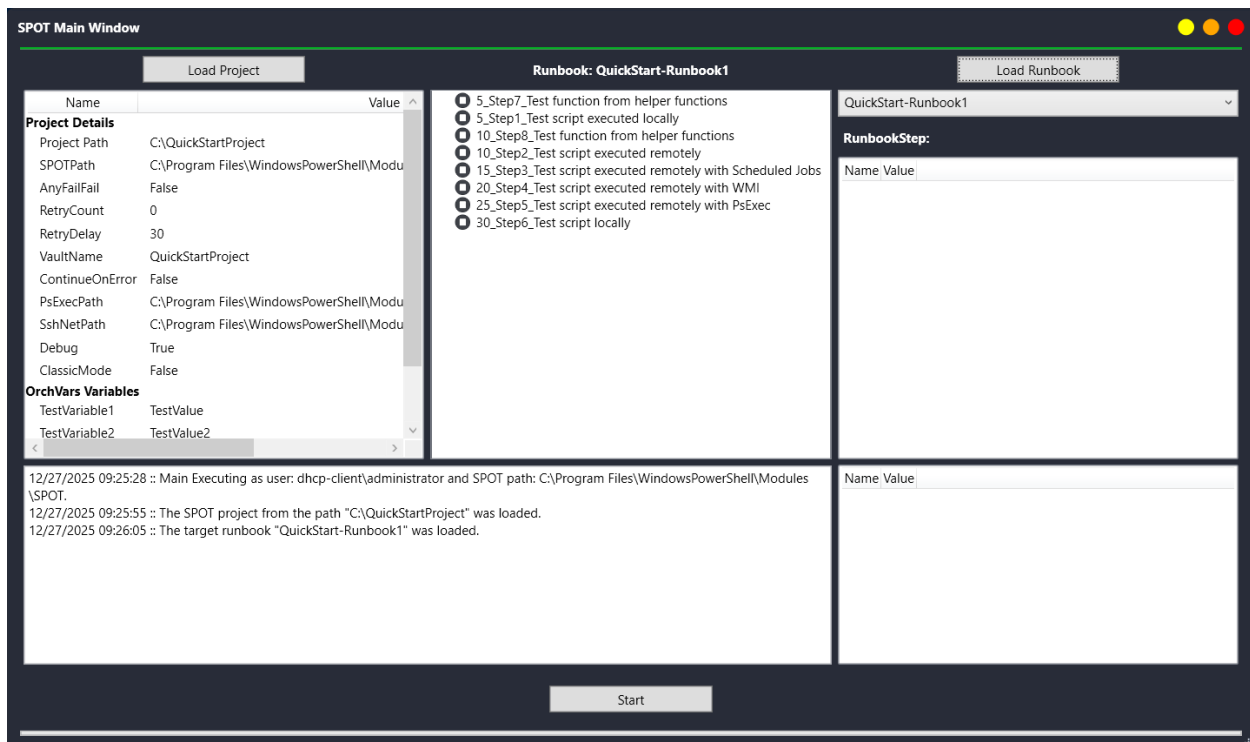
To execute the SPOT runbook from our project with GUI, the following command can be used:

```
Start-SPOTGUI
```

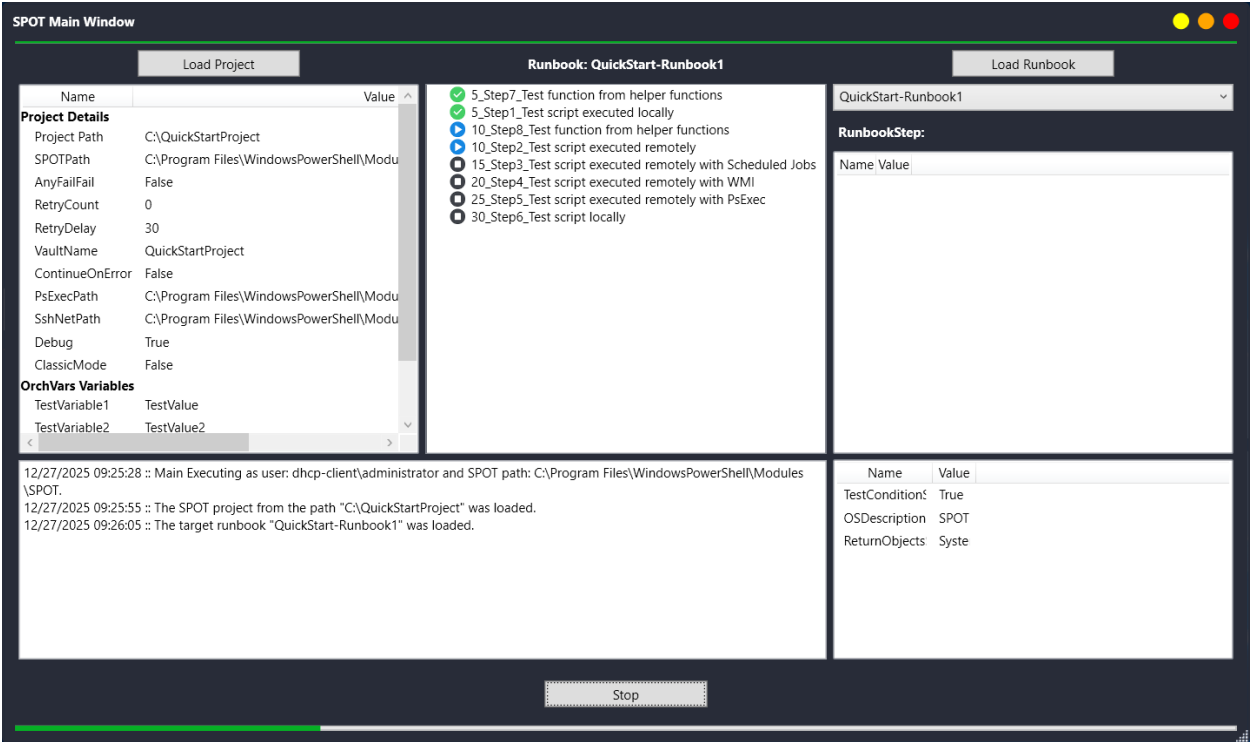
The SPOT GUI opens empty and looks like in the picture below.



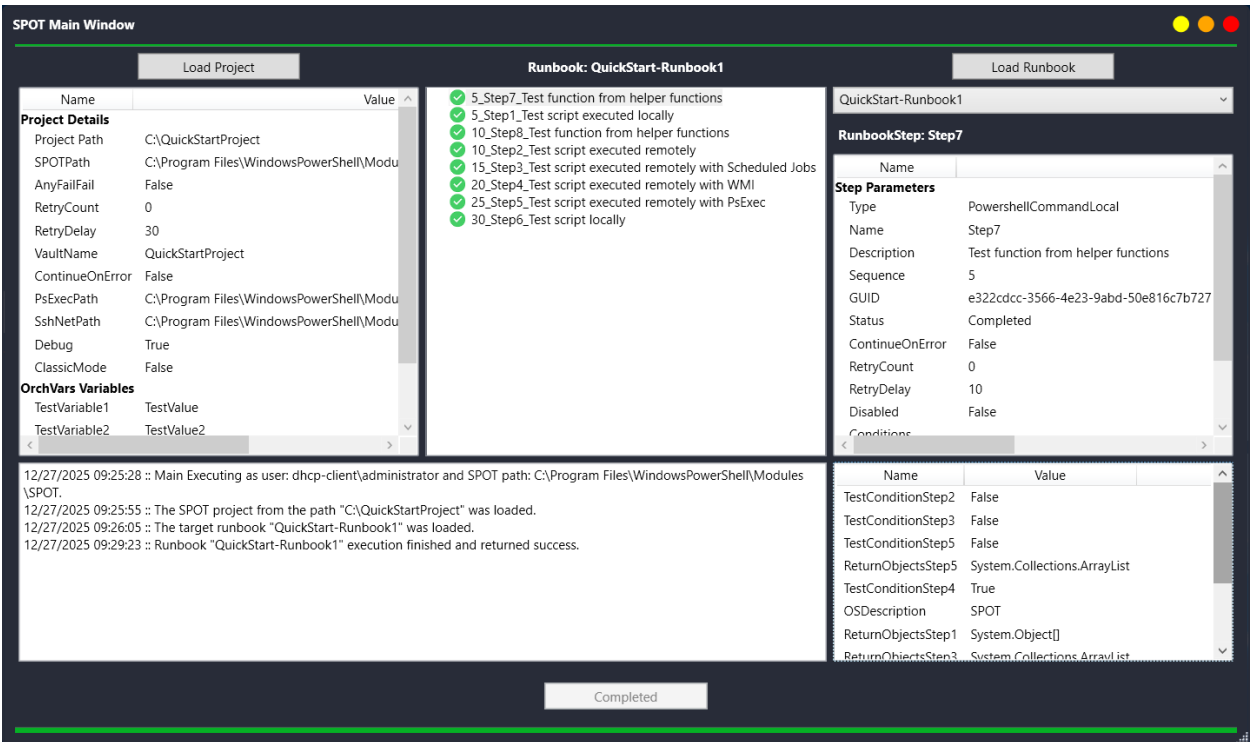
The project that we created can be loaded with the “Load Project” button and afterwards, the runbook can be loaded with the “Load Runbook” button. The GUI will then reflect the main elements from the project as well as from the loaded runbook, as seen in the picture below.



At this point, the runbook execution can be triggered by pressing the button “Start”. This is how the GUI reflects the execution progress.



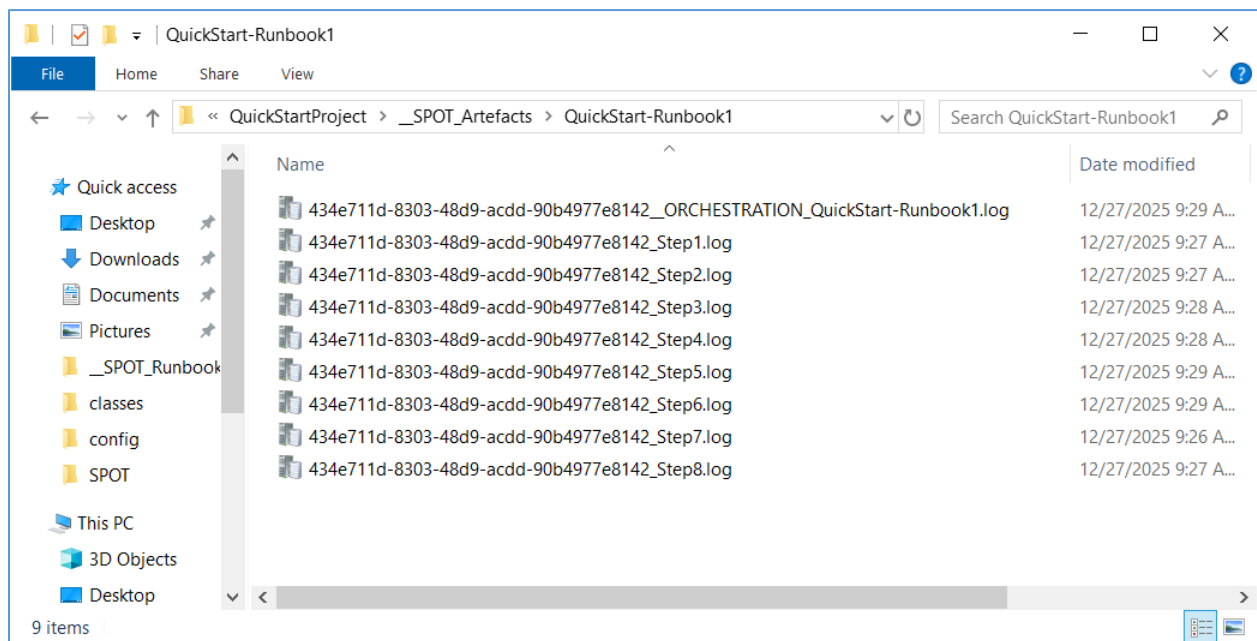
And this is how it looks like at the end, if all steps were successful.



3.4 Check the artefacts after execution

After the runbook has finished the execution, there are two places to check for execution artefacts. **First location**, in the project folder “__SPOT_Artefacts” where all the output from each step (including warnings and errors) is present in its own log file and the files are organized in folders representing the runbooks they belong to. Each runbook has also a runbook execution log file that contains internal SPOT logging. **Second location**, inside the PowerShell window, the internal SPOT objects are still available, like, for example, the \$PublishedData variable that contains all variables that have been published by all the steps in the orchestration.

In the picture below the resulting log file artefacts from the runbook execution can be seen. Each filename contains the internal SPOT GUID of the runbook they belong to and the step names.



Our runbook executed in Step6 a step script that exports the generated test objects from all steps types, local and remote, in a csv file inside the Project folder itself. We can check this file to make sure the objects were indeed generated, published and used from step to step until the export has been done in Step6, demonstrating the SPOT capacity to work with custom objects on remote computers using any of its remote execution methods.

3.5 Modify and test to showcase more key SPOT features

The initial test runbook that we executed previously is nice and fine, but it may be improved by parallelization. If we look closer, some actions are independent on the others, allowing for parallel execution. We could execute individual steps in parallel by setting the “Seq” values with the same

numbers, but we can also define new, smaller runbooks and execute them in parallel by setting the same “Seq” values as for the steps.

The runbook files below are a modified version of the initial runbook, with two runbooks (“OS-Description-Runbook” and “Remote-Custom-Objects-Runbook”) being independent and used as runbook steps in a parent runbook, “QuickStart-Runbook2”. They are not set to run in parallel on Seq 10, which would be also ok, but instead on Seq 5, as here they are in parallel with another local step, showcasing parallelization on the same target. So now, the second version of the initial runbook can be executed to demonstrate the runbook nesting and parallelization.

Finally, a third small change can be made to the “QuickStart-Runbook2” runbook to demonstrate remote runbook nesting, the execution of a child runbook on a remote target computer. For this, the Seq number of the “OS-Description-Runbook” step inside the “QuickStart-Runbook2” runbook must be changed to 10, to allow the first Step1 with Seq 5 to be finished and its published variables available at the time the child runbook “OS-Description-Runbook” is prepared and sent for execution on a remote computer. To configure the remote execution on the child runbook “OS-Description-Runbook” there are some commented out step parameters that need to be enabled. They specify mainly on which target computer to send the runbook for execution, with that credentials and with what remote execution method (could be any of them). The location of all these changes are highlighted with yellow in the content below.

OS-Description-Runbook.yaml

#####

Description: OS-Description-Runbook for learning/testing

RunbookSteps:

Step7:

Type: PowershellCommandLocal

Description: Test function from helper functions

Seq: 5

StepParameters:

CommandName: Get-WinOSDescription

VariablesToPublish: Description=OSDescription

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step8:

Type: PowershellCommandLocal

Description: Test function from helper functions

Seq: 10

StepParameters:

CommandName: Set-WinOSDescription

CommandParameters:

Description: SPOT

Conditions: \$PV:TestConditionStep1

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

#####

Remote-Custom-Objects-Runbook.yaml

#####

Description: Remote-Custom-Objects-Runbook for learning/testing

RunbookSteps:

Step2:

Type: PowershellCommandRemote

Description: Test script executed remotely

Seq: 10

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep1

ObjectCount: 10

ObjectClass: RemoteObject

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep2

- TestCondition=TestConditionStep2

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step3:

Type: PowershellCommandRemoteSJ

Description: Test script executed remotely with Scheduled Jobs

Seq: 15

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep2

ObjectCount: 10

ObjectClass: RemoteObjectSJ

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep3

- TestCondition=TestConditionStep3

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step4:

Type: PowershellCommandRemoteWMI

Description: Test script executed remotely with WMI

Seq: 20

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep3

ObjectCount: 10

ObjectClass: RemoteObjectWMI

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep4

- TestCondition=TestConditionStep4

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

Step5:

Type: PowershellCommandRemotePsExec

Description: Test script executed remotely with PsExec

Seq: 25

StepParameters:

CommandName: Generate-TestData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep4

ObjectCount: 10

ObjectClass: RemoteObjectPsExec

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep5

- TestCondition=TestConditionStep5

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

#####

QuickStart-Runbook2.yaml

#####

Description: QuickStart-Runbook2 for learning/testing

RunbookSteps:

Step1:

Type: PowershellCommandLocal

Description: Test script executed locally

Seq: 5

StepParameters:

CommandName: Generate-TestData

CommandParameters:

ObjectCount: 10

ObjectClass: LocalObject5

VariablesToPublish:

- ReturnObjects=ReturnObjectsStep1

- TestCondition=TestConditionStep1

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

OS-Description-Runbook:

Type: Runbook

Description: Test runbook runbookstep 1

Seq: 5

ContinueOnError: false

Disabled: false

#RemoteParameters:

ExecFunction: PowershellCommandRemote

RemoteComputer: 192.168.99.20

Credential: \$SV:SrvCred2

Remote-Custom-Objects-Runbook:

Type: Runbook

Description: Test runbook runbookstep 2

Seq: 10

ContinueOnError: false

Disabled: false

Step6:

Type: PowershellCommandLocal

Description: Test script locally

Seq: 30

StepParameters:

CommandName: Export-GeneratedData

CommandParameters:

InputObjects: \$PV:ReturnObjectsStep5

ObjectCount: 10

ObjectClass: LocalObject30

ExportPath: \$OV:_ProjectPath:\$OV\SPOTExportedObjects.csv

RetryCount: 2

RetryDelay: 10

ContinueOnError: false

Disabled: false

#####

Many other features are available in SOPT, but for the moment, for this short guide, we stop here with the examples. The features in this examples and the extra features not presented here are not mandatory in a SPOT project. You can start small and learn progressively over time.